1 Introduction

Image classification is a common problem in machine learning, where the goal is to predict a label for an image based on its visual content. One interesting aspect of this problem is that the difficulty can vary a lot depending on how similar the classes are to each other.

This project explores this through two classification tasks. Task 1 asks us to classify images into ten broad animal categories, such as cats, elephants, spiders, and horses. These categories look quite different from each other, so we expected standard machine learning methods to work reasonably well. Task 2 asks us to classify images into ten bird species from the Caltech-UCSD Birds-200-2011 (CUB-200-2011) dataset (Wah et al., 2011). This is much harder because many of the bird species look very similar, differing only in small details like beak shape or wing markings.

The main goal of this project is to compare different feature representations and classifiers across both tasks, and to understand why some approaches work better than others. We started with basic handcrafted features such as colour histograms and Histogram of Oriented Gradients (HOG) (Dalal and Triggs, 2005), and then moved to features extracted from a pretrained deep learning model (EfficientNet (Tan and Le, 2019)) to see how much of a difference feature quality makes.

This report is structured as follows. Section 2 describes our methodology, including how we preprocessed data, engineered features, and selected models. Section 3 presents the results. Section 4 discusses what we found and why. Section 5 concludes the report.

2 Methodology

2.1 Data Preprocessing

For both tasks, we loaded the metadata CSV files and the feature CSV files separately, then merged them using `image_id` as the key. After merging, we removed four columns that are not useful as features: `image_id`, `image_path`, `class_id`, and `class_name`. The remaining numeric columns were used as model input, and `class_name` was used as the label.

We split the data into training and validation sets using stratified sampling, so that each split has a similar proportion of each class. Task 1 used an 80/20 split, and Task 2 used a 75/25 split. For models sensitive to feature scale — Logistic Regression, Linear SVM, and RBF SVM — we applied `StandardScaler` fitted on the training set only. This avoids data leakage into the validation set. Random Forest does not require scaling, so it was trained on raw features.

2.2 Feature Representations

2.2.1 Baseline Features

The provided feature files gave us three types of features:

- **Colour histograms**: capture the distribution of colours across RGB channels.
- **HOG-PCA features**: capture edge and texture information, reduced via PCA.
- **Additional features**: basic image statistics such as edge density, texture variance, and average channel values.

We concatenated all three into a single feature vector. This formed the baseline representation for all initial experiments.

2.2.2 Feature Engineering

We explored three additional feature engineering approaches on top of the baseline:

HSV colour features: We extracted HSV channel statistics (mean, standard deviation, and histogram bins for each channel) from each image and added them to the baseline features. HSV separates colour (hue) from brightness and saturation, which we thought might provide more useful colour information than RGB alone.

Feature selection: We used `SelectKBest` with ANOVA F-scores to rank features by how well they separate the classes. We then tested different values of k from 10 to 220 and selected the best k based on validation accuracy. The best values were $k = 70$ for Task 1 and $k = 30$ for Task 2, as shown in Figure 1.

Background-suppressed texture features: We applied GrabCut-based background suppression before extracting texture and colour features. The idea was to focus on the foreground subject rather than irrelevant background pixels. We then combined this with feature selection to get our best handcrafted feature pipeline.

2.2.3 EfficientNet Deep Features

For our final models, we used pretrained EfficientNet models from ImageNet as feature extractors. We removed the final classification layer and used the penultimate layer output as a feature vector. Importantly, neither model

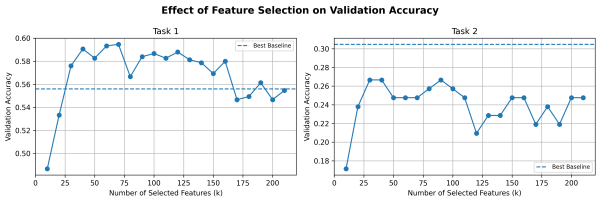


Figure 1: Effect of feature selection (k) on validation accuracy for Task 1 (left) and Task 2 (right). The dashed line shows the best baseline accuracy. For Task 1, $k = 70$ gives the best result; for Task 2, $k = 30$.

was trained on CIFAR-10 or CUB-200-2011, so using them as general feature extractors is allowed.

For Task 1, we used EfficientNet-B4, which outputs a 1792-dimensional feature vector. For Task 2, we used EfficientNet-B2 (a lighter model suited to the smaller dataset), which outputs a 1408-dimensional vector. We applied Test-Time Augmentation (TTA) — averaging features over multiple augmented versions of each image — to make the features more stable. For Task 2, we additionally applied PCA to reduce dimensionality to 200 components, which retained 93.1% of the variance and helped with the small dataset size.

2.3 Classification Models

2.3.1 Baseline Models

We evaluated four classifiers on the handcrafted features:

- **Logistic Regression**: a linear model that works well when classes are linearly separable. We set `max_iter=2000` to ensure convergence.
- **Linear SVM**: another linear model, often more robust than logistic regression in high-dimensional spaces (Cortes and Vapnik, 1995).
- **RBF SVM**: uses a radial basis function kernel to model non-linear decision boundaries.
- **Random Forest**: an ensemble of decision trees (Breiman, 2001). We included this as a tree-based comparison.

All four models were evaluated for Task 1. The same three linear/kernel models were used for Task 2 baselines, as Random Forest performed poorly in early experiments.

2.3.2 Hyperparameter Tuning

For Task 2, we applied `GridSearchCV` with 3-fold stratified cross-validation to tune the Random Forest model (Pedregosa et al., 2011). The search covered `n_estimators` (100, 200), `max_depth` (None, 10, 20), `min_samples_leaf` (1, 2, 4), and `max_features` (sqrt, 0.5). Macro F1-score was used as the optimisation metric.

2.3.3 EfficientNet Models and Ensemble

For Task 1 with EfficientNet features, we evaluated RBF SVM with $C = 20$ and $C = 50$, and Logistic Regression. For Task 2, we evaluated Linear SVM with $C = 1$ and $C = 5$, RBF SVM, and Logistic Regression with $C = 1$ and $C = 5$.

The final model for each task was a **weighted soft-voting ensemble**, where each classifier contributes probability predictions, and the final prediction is the weighted average. The weight for each model was set proportional to its 5-fold cross-validation accuracy. This ensemble approach is our required ensemble model for Task 2.

2.4 Evaluation

All models were evaluated using accuracy, macro precision, macro recall, and macro F1-score on the validation set. Macro averaging treats all classes equally regardless of size, which is appropriate since both tasks have roughly balanced classes. We also used confusion matrices and per-class F1 scores to identify which classes were hardest to classify.

3 Results

3.1 Baseline Model Performance

Table 1 and Table 2 show the baseline results for Task 1 and Task 2 respectively.

Model	Train Acc.	Val. Acc.	Prec.	Recall	F1	Low F1	Class
Linear SVM	0.731	0.556	0.549	0.556	0.551	0.361	cat
Logistic Regression	0.765	0.545	0.542	0.545	0.542	0.386	cat
RBF SVM	0.902	0.512	0.531	0.512	0.517	0.368	cat
Random Forest	1.000	0.515	0.512	0.515	0.509	0.321	deer

Table 1: Task 1 baseline model performance on the validation set.

Model	Train Acc.	Val. Acc.	Prec.	Recall	F1	Low F1	Class
Linear SVM	1.000	0.305	0.319	0.306	0.302	0.000	Wilson Warbler
Logistic Regression	1.000	0.267	0.290	0.267	0.262	0.000	Wilson Warbler
RBF SVM	0.904	0.238	0.315	0.243	0.207	0.000	Herring Gull

Table 2: Task 2 baseline model performance on the validation set.

For Task 1, Linear SVM performed best at 55.6% validation accuracy and a macro F1 of 0.551. For Task 2, all baseline models did

poorly, with the best being Linear SVM at 30.5% accuracy. The most striking observation is that all Task 2 models achieved 0.000 F1 on at least one class (Wilson Warbler or Herring Gull), meaning those classes were never predicted correctly.

Figure 2 and Figure 3 show the confusion matrices for the best baseline model on each task.



Figure 2: Task 1 baseline confusion matrix (Linear SVM).

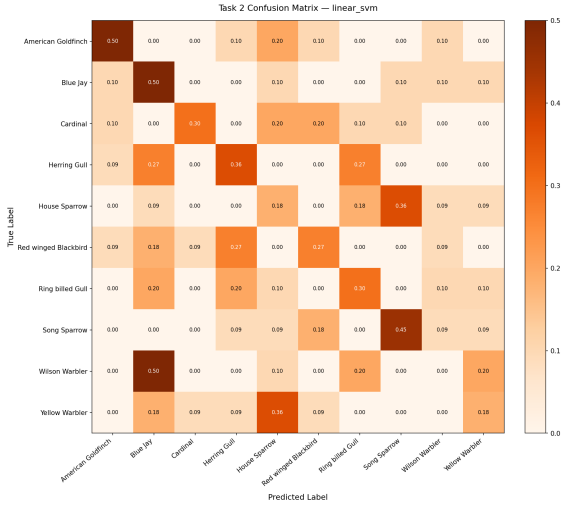


Figure 3: Task 2 baseline confusion matrix (Linear SVM).

3.2 Feature Engineering Results

Table 3 summarises the best F1 score from each feature engineering experiment.

For Task 1, feature selection and background suppression both helped. The combination of background suppression with feature selection gave the best handcrafted result of F1 = 0.606. However, for Task 2, none of the feature engineering approaches consistently beat the baseline. HSV features actually made things worse for Task 1 on their own (likely because they replaced rather than supplemented the baseline).

Task	Experiment	Best Model	Val. F1
Task 1	Baseline	Linear SVM	0.551
Task 1	HSV features	RBF SVM	0.246
Task 1	Feature selection ($k = 70$)	Logistic Regression	0.592
Task 1	Background suppression	Linear SVM	0.568
Task 1	BG suppression + selection	Logistic Regression	0.606
Task 2	Baseline	Linear SVM	0.302
Task 2	HSV features	RBF SVM	0.263
Task 2	Feature selection ($k = 30$)	Linear SVM	0.287
Task 2	Background suppression	Logistic Regression	0.278
Task 2	BG suppression + selection	Linear SVM	0.274

Table 3: Comparison of feature engineering experiments.

3.3 Hyperparameter Tuning Results

For Task 2, the best Random Forest parameters found by grid search were: `max_depth=None`, `max_features=0.5`, `min_samples_leaf=4`, `n_estimators=100`, with a best cross-validation macro F1 of 0.286. On the validation set, this tuned model achieved 31.4% accuracy and F1 = 0.293. This is slightly better than the untuned baseline but still far below the EfficientNet models.

3.4 EfficientNet-Based Model Results

Table 4 and Table 5 show the 5-fold cross-validation results for EfficientNet-based models.

Model	CV Acc.	Std.
RBF SVM ($C = 20$)	0.9259	0.0098
RBF SVM ($C = 50$)	0.9259	0.0098
Logistic Regression	0.9253	0.0111

Table 4: Task 1 EfficientNet-B4 5-fold cross-validation results.

Model	CV Acc.	Std.
Linear SVM ($C = 1$)	0.9305	0.0174
Linear SVM ($C = 5$)	0.9305	0.0174
RBF SVM	0.9065	0.0336
Logistic Regression ($C = 1$)	0.9186	0.0202
Logistic Regression ($C = 5$)	0.9186	0.0202

Table 5: Task 2 EfficientNet-B2 5-fold cross-validation results.

All EfficientNet-based models achieved over 90% cross-validation accuracy on both tasks. The weighted soft-voting ensemble achieved 90.5% accuracy on a held-out validation split

for Task 2. Figure 4 shows the confusion matrix for this ensemble.

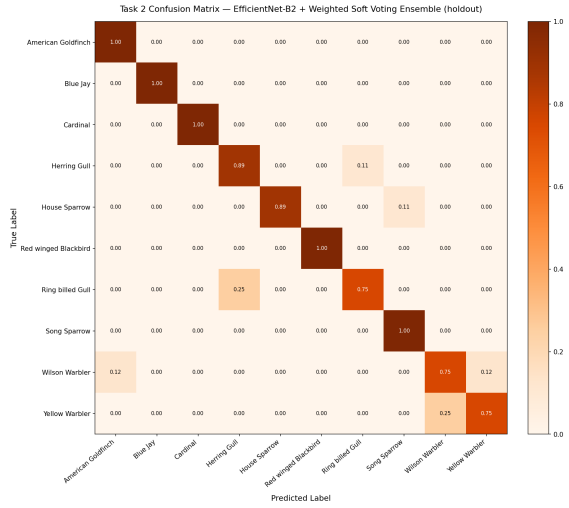


Figure 4: Task 2 EfficientNet-B2 ensemble confusion matrix (holdout validation).

3.5 Overall Comparison

Table 6 summarises all major experiments across both tasks.

Experiment	Val. / CV Acc.	Task
Task 1 — Linear SVM (baseline)	0.556	Task 1
Task 1 — Logistic Regression (baseline)	0.545	Task 1
Task 1 — RBF SVM (baseline)	0.512	Task 1
Task 1 — Random Forest (baseline)	0.515	Task 1
Task 1 — BG suppression + selection	0.607	Task 1
Task 1 — EfficientNet-B4 + Ensemble	0.926	Task 1
Task 2 — Linear SVM (baseline)	0.305	Task 2
Task 2 — Logistic Regression (baseline)	0.267	Task 2
Task 2 — Tuned Random Forest	0.314	Task 2
Task 2 — EfficientNet-B2 + Ensemble	0.905	Task 2

Table 6: Summary of all major experiments.

Figure 5 provides a bar chart comparing all experiments.

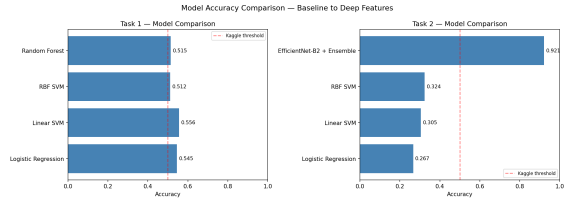


Figure 5: Validation/CV accuracy comparison across all experiments and both tasks.

4 Discussion

4.1 Feature Quality Mattered More Than Model Choice

One of the clearest things we found is that changing the features had a much bigger effect on performance than changing the classifier. Within the handcrafted feature experiments, switching from Logistic Regression to Linear SVM to RBF SVM only changed accuracy by a few percentage points. But switching from handcrafted features to EfficientNet features increased accuracy from around 55% to over 92% on Task 1, and from 30% to over 90% on Task 2.

This makes sense when we think about what these features actually represent. HOG features describe local edge orientations, and colour histograms describe overall colour distributions. These are low-level properties that can help tell apart very different categories (like elephant vs. butterfly), but they are not able to capture the more specific patterns needed to tell apart similar-looking birds. EfficientNet, on the other hand, was trained on millions of images from ImageNet and has learned to extract hierarchical visual patterns, from edges and textures at early layers to object parts and full shapes at later layers (Tan and Le, 2019). These richer representations gave the classifiers much more to work with.

This finding is consistent with the general shift in computer vision from handcrafted features to learned representations, and it matches what is discussed in lectures about the importance of the hypothesis space and feature engineering.

4.2 Why Task 2 Was Much Harder

The difference in difficulty between the two tasks was very clear. The best handcrafted baseline on Task 1 reached 55.6% accuracy, while on Task 2 it only reached 30.5%. Even the tuned Random Forest on Task 2 only reached 31.4%.

The main reason is the nature of the classes. Task 1 classes like elephant, spider, horse, and butterfly differ in overall shape, colour, and size. Even simple colour histograms can separate many of these. Task 2 classes are all birds, so they share the same basic body shape. The differences between, say, a House Sparrow and a Song Sparrow are mostly in fine-grained details — subtle streaking patterns on the chest, slight differences in beak shape — that are not captured by low-level features.

Dataset size also played a role. Task 1 had about 4,000 training images across 10 classes (400 per class), while Task 2 had only around 350–400 total (35–40 per class). With so few

samples per class, models have less data to learn from and are more likely to overfit. This is visible in Table 2: all three baseline models achieved 100% or near-100% training accuracy but much lower validation accuracy, showing severe overfitting. By contrast, in Task 1, the overfitting gap was smaller, especially for the linear models.

4.3 Feature Engineering: What Helped and What Did Not

Among the handcrafted feature experiments, feature selection (using ANOVA F-scores to pick the top k features) was the most consistently useful. For Task 1, it improved F1 from 0.551 to 0.592. The reason is that the full feature set likely contains many features that are noisy or irrelevant, and removing them helps the model focus on the more informative ones. The best k for Task 1 was 70 out of over 200 features, suggesting that many of the provided features were not that useful on their own.

Background suppression also helped a little for Task 1 (F1 improved from 0.551 to 0.568). The idea was that removing background pixels might help the model focus on the animal itself. However, the improvement was modest, probably because the GrabCut algorithm does not always correctly separate the foreground from the background, especially for small or camouflaged animals.

For Task 2, none of the feature engineering approaches helped much. This suggests that the handcrafted features simply do not contain enough information to distinguish between bird species, regardless of how we process them. The problem is a representation problem rather than a noise or dimensionality problem.

Interestingly, HSV features alone (without the original baseline features) performed much worse than the baseline for Task 1. This is because we treated them as a standalone feature set rather than adding them to the existing features, which removed the useful HOG and histogram features. When we added them on top of the baseline, the improvement was also minimal, suggesting that the baseline colour features already captured most of the useful colour information.

4.4 Error Analysis

4.4.1 Task 1 Error Analysis

Looking at the Task 1 confusion matrix (Figure 2), the most common confusions were between dog and cat (15 errors each way), and between sheep and elephant (13 errors each way). These pairs make intuitive sense: dogs and cats have similar body shapes and fur textures, and in small 64×64 images it can be hard to tell them apart. Sheep and elephant are more surprising, but may be explained by similar grey/white colouring and similar body proportions at this resolution.

The class with the lowest F1 across most models was *cat* (F1 around 0.36–0.42). This is consistent with the dog-cat confusion above: cats were frequently predicted as dogs and vice versa, pulling down both classes' F1 scores.

4.4.2 Task 2 Baseline Error Analysis

For Task 2, the most common confusions under the baseline Linear SVM were Wilson Warbler being predicted as Blue Jay (5 times), House Sparrow being predicted as Song Sparrow (4 times), and Yellow Warbler being predicted as House Sparrow (4 times). The fact that Wilson Warbler was most often predicted as Blue Jay (rather than as another warbler) suggests the model is not capturing any meaningful semantic similarity — it is essentially guessing. Wilson Warbler and Herring Gull both had F1 = 0 under all baseline models, meaning neither class was ever predicted correctly. These are likely the hardest classes because they appear relatively infrequently in the training data and share many visual features with other classes.

4.4.3 Task 2 EfficientNet Error Analysis

With EfficientNet features, the errors became much more interpretable. The remaining confusions were between visually similar pairs: Yellow Warbler vs Wilson Warbler (2 errors each way), Ring-billed Gull vs Herring Gull (2 errors), and House Sparrow vs Song Sparrow (1 error). These are exactly the pairs we would expect to be hardest, since they share the most visual similarity. Both warbler species are small yellow birds, with the main difference being the Wilson Warbler's black cap. The two gull species differ mainly in bill ring pattern, which is a very small detail in a 128×128 image. These results suggest that even good deep features still struggle with the hardest fine-grained pairs, and that more specialised methods (such as attention to specific parts of the bird) might be needed to further improve performance.

4.5 Overfitting and Model Behaviour

Random Forest showed the most severe overfitting across both tasks: it consistently reached

100% training accuracy but performed no better than linear models on validation. This is a known issue with Random Forest — it can perfectly memorise the training data by growing very deep trees, but this does not generalise well, especially when training data is small (Task 2). The linear models (Logistic Regression and Linear SVM) had smaller overfitting gaps and more stable performance, which is consistent with their simpler decision boundaries and built-in regularisation.

The ensemble models helped slightly for Task 2 with EfficientNet features, but the improvement over individual models was small. This is expected when all models are already performing similarly — there is less benefit from combining them because they tend to agree on most predictions.

5 Conclusions

This project compared handcrafted and deep learning features for coarse-grained and fine-grained image classification. The main findings are as follows.

Feature representation was the most important factor. Switching from handcrafted features (HOG, colour histograms) to EfficientNet deep features improved accuracy from around 55% to over 92% on Task 1, and from 30% to over 90% on Task 2. No amount of feature engineering on the handcrafted features could close this gap, especially for Task 2.

The two tasks differed greatly in difficulty. Task 1 was manageable with basic methods because the classes are visually distinct. Task 2 was much harder because the bird species share similar appearance, and the training dataset is very small. This matches the challenge described in fine-grained visual recognition literature.

Among the classifiers, linear models were the most reliable. Random Forest consistently overfit, especially on Task 2. The weighted soft-voting ensemble performed best overall by combining predictions from multiple classifiers.

Error analysis showed that the remaining mistakes, even with EfficientNet features, mostly involved pairs of visually similar species (the two warblers and the two gulls). This suggests that further improvement would require either more training data, higher-resolution images, or part-based models that explicitly focus on discriminative regions.

References

Leo Breiman. 2001. Random forests. *Machine Learning*, 45:5–32.

Corinna Cortes and Vladimir Vapnik. 1995. Support-vector networks. *Machine Learning*, 20:273–297.

Navneet Dalal and Bill Triggs. 2005. Histograms of oriented gradients for human detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, volume 1, pages 886–893.

Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. 2011. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830.

Mingxing Tan and Quoc V. Le. 2019. Efficientnet: Rethinking model scaling for convolutional neural networks. In *Proceedings of the 36th International Conference on Machine Learning*, pages 6105–6114. PMLR.

Catherine Wah, Steve Branson, Peter Welinder, Pietro Perona, and Serge Belongie. 2011. The caltech-UCSD birds-200-2011 dataset. Technical Report CNS-TR-2011-001, California Institute of Technology.